In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

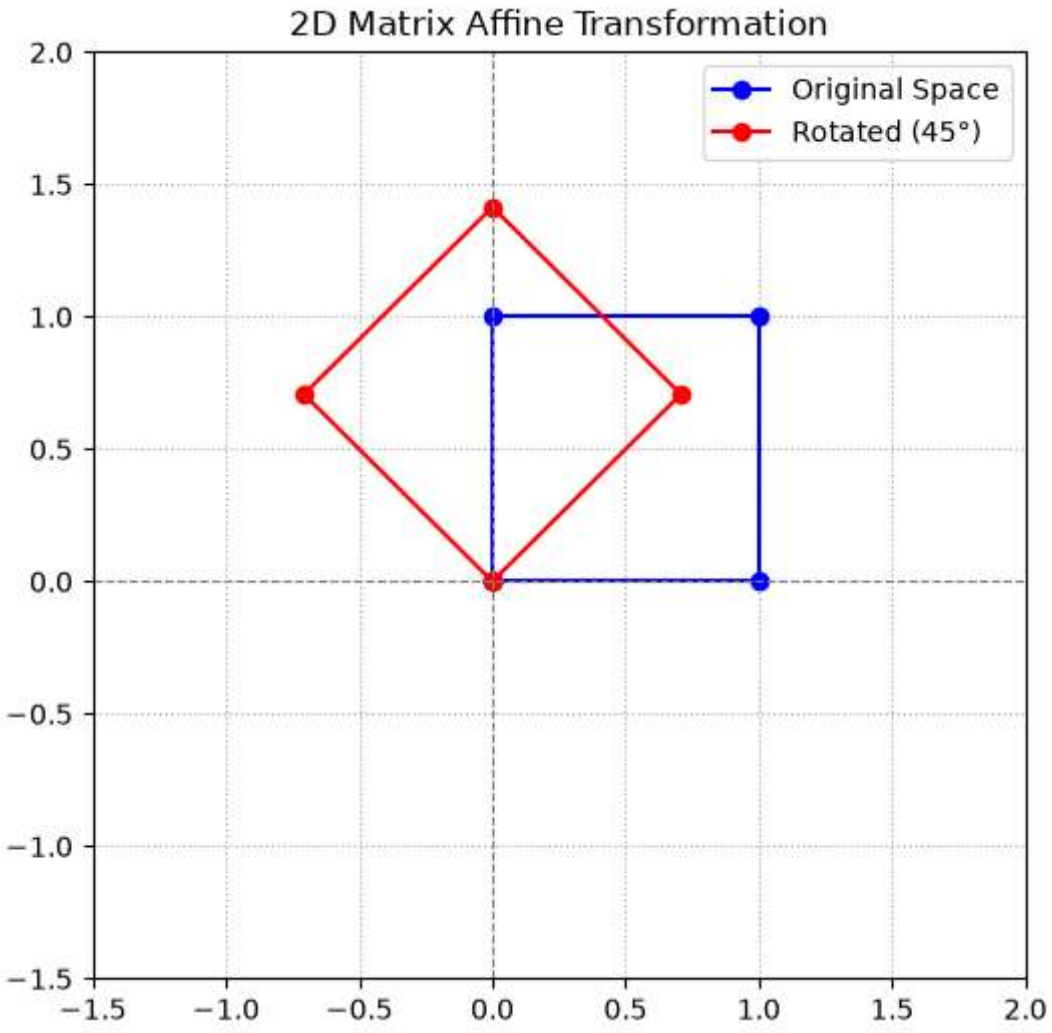
# 1. Vector Operations
v1 = np.array([2.0, 3.0])
v2 = np.array([4.0, 1.0])

dot_product = np.dot(v1, v2)
outer_product = np.outer(v1, v2)
l2_norm = np.linalg.norm(v1)

print(f"Dot Product: {dot_product}")
print(f"Outer Product:\n{outer_product}")
print(f"L2 Norm of v1: {l2_norm:.4f}\n")
W = np.array([[0.8, -0.5],
              [0.3,  0.9],
              [-0.2, 0.4]])
# Batch of 4 inputs (2 features each)
X = np.array([[1.0, 2.0],
              [3.0, 4.0],
              [-1.0, 0.5],
              [2.0, -1.5]]).T  # Shape: (2, 4)
# Bias vector (3, 1)
b = np.array([[0.1],
              [-0.2],
              [0.05]])
Z = np.dot(W, X) + b
print(f"Linear Output Shape: {Z.shape}")
print(f"Z:\n{Z}\n")
theta = np.pi / 4  # 45 degrees
A_rot = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta),  np.cos(theta)]])
pts = np.array([[0, 0], [1, 0], [1, 1], [0, 1], [0, 0]]).T
transformed_pts = A_rot @ pts
plt.figure(figsize=(6, 6))
plt.plot(pts[0, :], pts[1, :], 'b-o', label='Original Space')
plt.plot(transformed_pts[0, :], transformed_pts[1, :], 'r-o', label='Rotated (45°)'
plt.axhline(0, color='gray', linestyle='--', linewidth=0.8)
plt.axvline(0, color='gray', linestyle='--', linewidth=0.8)
plt.grid(True, linestyle=':')
plt.xlim(-1.5, 2)
plt.ylim(-1.5, 2)
plt.title("2D Matrix Affine Transformation")
plt.legend()
plt.show()
```

```
Dot Product: 11.0
Outer Product:
[[ 8.  2.]
 [12.  3.]]
L2 Norm of v1: 3.6056

Linear Output Shape: (3, 4)
Z:
[[-0.1   0.5  -0.95  2.45]
 [ 1.9   4.3  -0.05 -0.95]
 [ 0.65  1.05  0.45 -0.95]]
```



In [2]:
```python
#2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Generate correlated 2D synthetic dataset
np.random.seed(42)
mean = [5.0, 10.0]
# Defined covariance structure
cov_true = [[3.0, 2.2],
            [2.2, 2.0]]
data = np.random.multivariate_normal(mean, cov_true, size=300)
```

```
df = pd.DataFrame(data, columns=['Feature_1', 'Feature_2'])

# Mean centering
X_centered = df.values - df.mean().values

# Sample Covariance Matrix: (X_c^T * X_c) / (N - 1)
N = X_centered.shape[0]
cov_matrix = (X_centered.T @ X_centered) / (N - 1)
print("Computed Covariance Matrix:\n", cov_matrix)

# Eigen-decomposition
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
# Sort in descending order
idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

print(f"\nEigenvalues:\n{eigenvalues}")
print(f"Eigenvectors:\n{eigenvectors}")

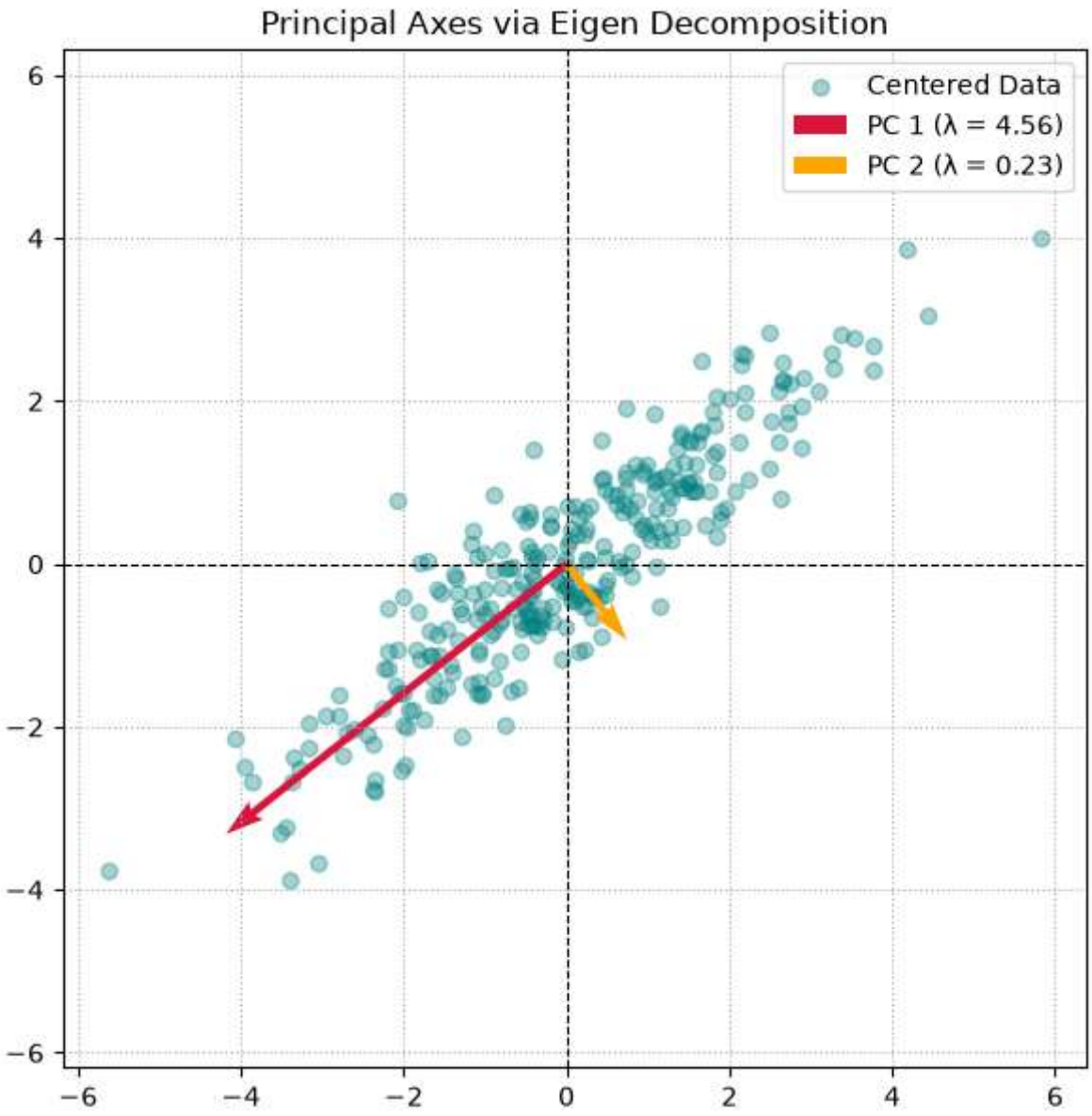
# Visualization of Eigenvectors over Data Points
plt.figure(figsize=(7, 7))
plt.scatter(X_centered[:, 0], X_centered[:, 1], alpha=0.35, color='teal', label='Ce

origin = [0, 0]
for i in range(len(eigenvalues)):
    v = eigenvectors[:, i] * np.sqrt(eigenvalues[i]) * 2.5
    plt.quiver(*origin, *v, angles='xy', scale_units='xy', scale=1,
               color='crimson' if i == 0 else 'orange',
               label=f'PC {i+1} (λ = {eigenvalues[i]:.2f})')

plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
plt.axvline(0, color='black', linewidth=0.8, linestyle='--')
plt.grid(True, linestyle=':')
plt.axis('equal')
plt.title("Principal Axes via Eigen Decomposition")
plt.legend()
plt.show()
```

```
Computed Covariance Matrix:
 [[2.88816248 2.10964247]
 [2.10964247 1.90083796]]

Eigenvalues:
[4.56113212 0.22786832]
Eigenvectors:
[[-0.78353295  0.62135023]
 [-0.62135023 -0.78353295]]
```



In [5]:
```python
#3
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

# 1. Normal Probability Density Function (PDF) & CDF
x = np.linspace(-6, 6, 1000)
pdf_vals = norm.pdf(x, loc=0, scale=1)
cdf_vals = norm.cdf(x, loc=0, scale=1)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(x, pdf_vals, 'b-', lw=2, label='Standard Normal PDF')
axes[0].fill_between(x, pdf_vals, where=(x >= -1) & (x <= 1), color='blue', alpha=0
axes[0].set_title("Gaussian Probability Density Function")
axes[0].grid(True, linestyle=':')
axes[0].legend()

# 2. Sequential Bayesian Updating (Coin Toss Simulation)
```

```
# True parameter: p = 0.7 (Biased coin)
true_p = 0.7
n_trials = [0, 2, 10, 50, 200]
np.random.seed(42)
flips = np.random.binomial(1, true_p, size=200)

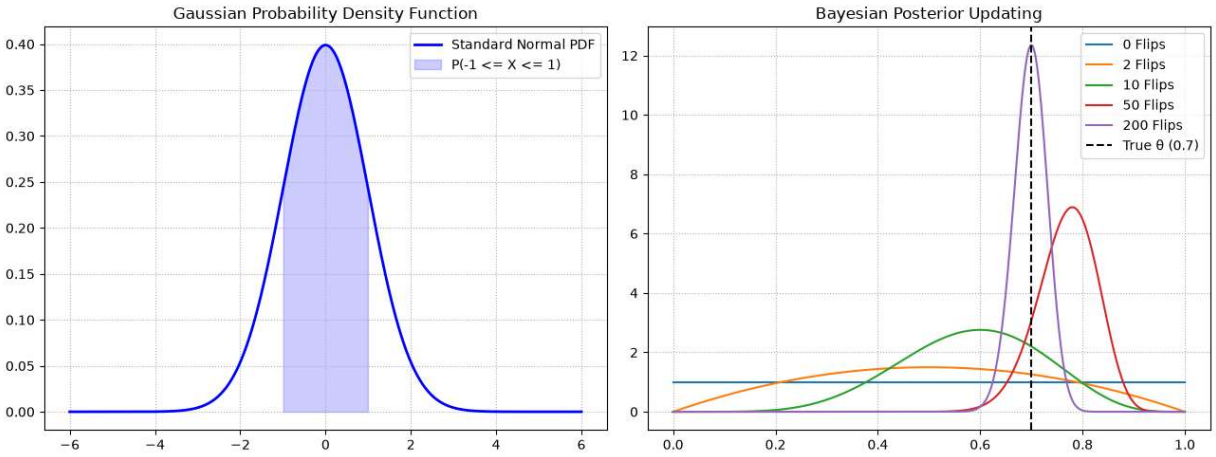
p_grid = np.linspace(0, 1, 500)
prior = np.ones_like(p_grid)  # Uniform Prior Beta(1, 1)

for n in n_trials:
    if n == 0:
        posterior = prior
    else:
        heads = np.sum(flips[:n])
        tails = n - heads
        # Likelihood * Prior (Analytical Conjugate Beta distribution: Beta(1+heads,
        likelihood = (p_grid ** heads) * ((1 - p_grid) ** tails)
        posterior = likelihood * prior
        posterior /= np.trapezoid(posterior, p_grid)  # Normalize

    axes[1].plot(p_grid, posterior, label=f'{n} Flips')

axes[1].axvline(true_p, color='black', linestyle='--', label=f'True θ ({true_p})')
axes[1].set_title("Bayesian Posterior Updating")
axes[1].grid(True, linestyle=':')
axes[1].legend()

plt.tight_layout()
plt.show()
```



In [6]:

```
#4
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import entropy as scipy_kl

def shannon_entropy(p, eps=1e-12):
    p = np.clip(p, eps, 1.0)
    return -np.sum(p * np.log2(p))

def cross_entropy(p, q, eps=1e-12):
    q = np.clip(q, eps, 1.0)
    return -np.sum(p * np.log(q))
```

```
def kl_divergence(p, q, eps=1e-12):
    p = np.clip(p, eps, 1.0)
    q = np.clip(q, eps, 1.0)
    return np.sum(p * np.log(p / q))

# Discrete categorical distributions
p = np.array([0.7, 0.2, 0.1])
q = np.array([0.4, 0.4, 0.2])

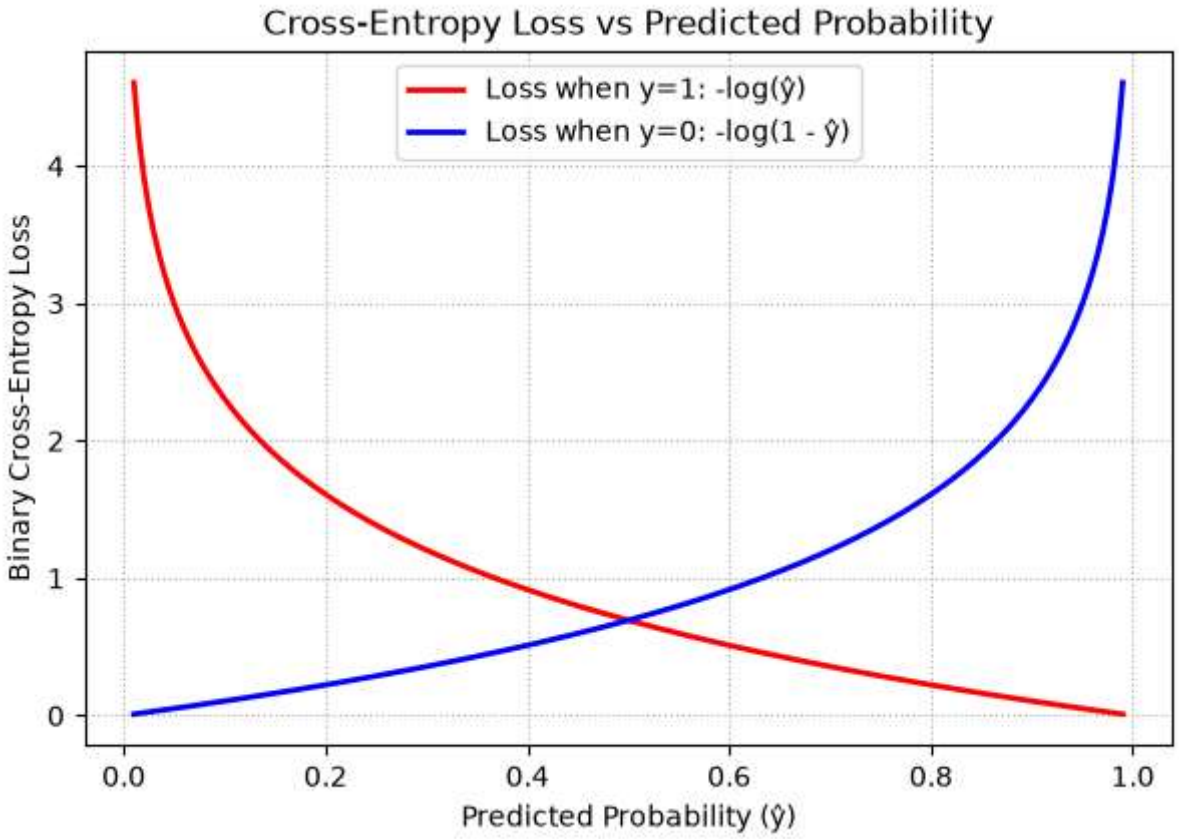
h_p = shannon_entropy(p)
ce_pq = cross_entropy(p, q)
kl_pq = kl_divergence(p, q)

print(f"Shannon Entropy H(P): {h_p:.4f} bits")
print(f"Cross-Entropy H(P, Q): {ce_pq:.4f} nats")
print(f"KL Divergence D_KL(P || Q): {kl_pq:.4f} nats")
print(f"Verification using Scipy: {scipy_kl(p, q):.4f} nats\n")

# Visualization: Binary Cross-Entropy Loss Curve
y_true = 1.0
y_pred = np.linspace(0.01, 0.99, 200)
bce_loss_1 = -np.log(y_pred)            # Loss when true label = 1
bce_loss_0 = -np.log(1 - y_pred)        # Loss when true label = 0

plt.figure(figsize=(7, 4.5))
plt.plot(y_pred, bce_loss_1, 'r-', lw=2, label='Loss when y=1: -log(ŷ)')
plt.plot(y_pred, bce_loss_0, 'b-', lw=2, label='Loss when y=0: -log(1 - ŷ)')
plt.xlabel("Predicted Probability (ŷ)")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Cross-Entropy Loss vs Predicted Probability")
plt.grid(True, linestyle=':')
plt.legend()
plt.show()
```

```
Shannon Entropy H(P): 1.1568 bits
Cross-Entropy H(P, Q): 0.9856 nats
KL Divergence D_KL(P || Q): 0.1838 nats
Verification using Scipy: 0.1838 nats
```



In [7]:
```python
#5
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Generate synthetic non-linear dataset
np.random.seed(42)
def true_fn(x):
    return np.sin(1.5 * np.pi * x)

n_samples = 30
x_train = np.sort(np.random.rand(n_samples))
y_train = true_fn(x_train) + np.random.normal(0, 0.15, n_samples)

x_val = np.sort(np.random.rand(n_samples))
y_val = true_fn(x_val) + np.random.normal(0, 0.15, n_samples)

x_plot = np.linspace(0, 1, 200)

# Evaluate Degrees representing Underfitting, Optimal, and Overfitting
degrees = [1, 4, 15]
titles = ['Underfitting (High Bias)', 'Good Fit (Balanced)', 'Overfitting (High Var

plt.figure(figsize=(14, 4))
for i, d in enumerate(degrees):
    ax = plt.subplot(1, 3, i + 1)
```

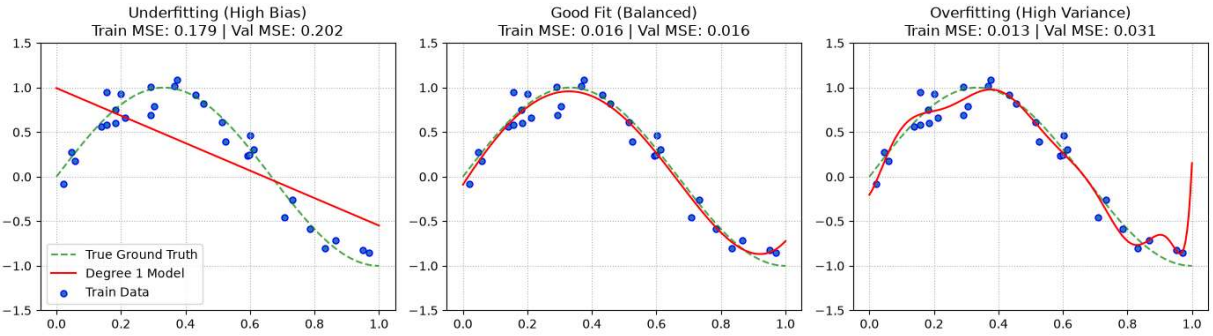
```
    model = make_pipeline(PolynomialFeatures(degree=d), LinearRegression())
    model.fit(x_train[:, np.newaxis], y_train)

    y_pred_plot = model.predict(x_plot[:, np.newaxis])

    # Calculate errors
    train_mse = mean_squared_error(y_train, model.predict(x_train[:, np.newaxis]))
    val_mse = mean_squared_error(y_val, model.predict(x_val[:, np.newaxis]))

    ax.plot(x_plot, true_fn(x_plot), 'g--', label='True Ground Truth', alpha=0.7)
    ax.plot(x_plot, y_pred_plot, 'r-', label=f'Degree {d} Model')
    ax.scatter(x_train, y_train, edgecolor='b', s=25, label='Train Data')
    ax.set_ylim(-1.5, 1.5)
    ax.set_title(f"{titles[i]}\nTrain MSE: {train_mse:.3f} | Val MSE: {val_mse:.3f}
    ax.grid(True, linestyle=':')
    if i == 0:
        ax.legend(loc='lower left')

plt.tight_layout()
plt.show()
```



In [ ]: